Chapter 7. AutoML and AutoGluon

Predictive Modeling, Lecture 6 (model selection, OOF predictions, fold bagging, stacking, weighted ensembles)

Contents

1 When There Are Many Candidates, Model Selection Becomes a Problem **2**

2 Stacking: Candidate Predictions Become New Inputs **2**

2.1 Why the level-1 inputs must be out-of-fold 2

3 K-fold Bagging and OOF Predictions **3**

4 Weighted Ensembles **3**

5 How the Three Mechanisms Fit Together in AutoGluon **4**

6 Setting the Search Space **4**

7 Responsible Use of AutoML **5**

1 When There Are Many Candidates, Model Selection Becomes a Problem

Candidates built in Lectures 1–5: regularized linear models; random forests and boosting; MLPs and tabular pre-trained models. No model is always best on every tabular data set. The decision an ensemble needs: *how should several candidates be combined?*

Stage	What AutoGluon does
fit candidates	trains several candidates: linear models, tree boosting, neural networks, ...
internal validation	non-bagged models on a holdout set, bagged models with OOF predictions
ensemble	combines candidates by fold bagging, stacking, weighted ensembles
leaderboard	compares internal validation score, fit time, prediction time, stack level
final prediction	predicts new cases with the selected single model or ensemble

AutoGluon is not a new algorithm but an AutoML system that **manages the fitting, validation and combination of many algorithms**. The validation principles of Ch. 1 remain the user's responsibility: AutoGluon's internal splits are random/stratified unless told otherwise, so time or group structure and a protected test set must be imposed from outside.

2 Stacking: Candidate Predictions Become New Inputs

For a case x_i, several candidate models each predict:

$$z_i = (h_{\text{linear}}(x_i),\ h_{\text{boost}}(x_i),\ h_{\text{MLP}}(x_i)), \qquad \text{e.g. } z_i = (0.30,\ 0.75,\ 0.60)$$

as binary-class probabilities. A higher-level model uses these predictions as new features:

$$\hat{y}_i = g(x_i, z_i),$$

where z_i are the candidates' predictions and g is a higher-level model that uses the original features and the candidates' predictions. Stacking does not pick the single best model; it **turns the predictions of several models into new features and learns again**, so it can exploit that candidates err on different cases (e.g. trust the MLP where the boosting model is uncertain).

2.1 Why the level-1 inputs must be out-of-fold

Theorem: in-sample stacking rewards overfitting

If g is trained on in-sample predictions $z_i = h(x_i)$ of models that were fitted on the same rows, a model that memorizes the training data (e.g. 1-nearest neighbor, whose in-sample prediction is exactly y_i) looks perfect, and g puts all its weight on it, although its out-of-sample error can be the worst of all candidates.

The training inputs of g must therefore have the same distribution as the inputs it will see at prediction time: predictions on cases *that the candidate did not see*. That is what OOF predictions provide.

3 K-fold Bagging and OOF Predictions

Definition: fold bagging

Split the training data into folds $F_1, \ldots, F_K$ and fit the same model K times, the kth time without F_k, giving $h^{(-1)}, \ldots, h^{(-K)}$ (same algorithm and settings, different training rows).

- **Training data: OOF predictions.** $i \in F_k \implies \hat{y}_i^{\text{OOF}} = h^{(-k)}(x_i)$: each case is predicted by the model that did not use it.
- **New cases: averaged prediction.** $x_{\text{new}} \mapsto \frac{1}{K}\sum_{k=1}^{K} h^{(-k)}(x_{\text{new}})$.

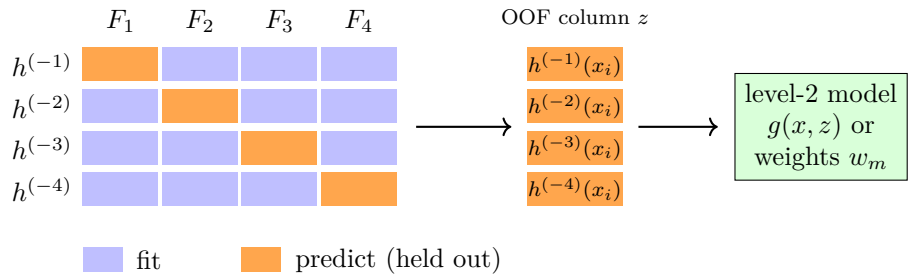


Figure 1: Fold bagging with $K = 4$. Model $h^{(-k)}$ is fitted without fold F_k and predicts only F_k; stacking the four held-out blocks gives an out-of-fold column z for every training row. The level-2 learner (stacker or weighted ensemble) is trained on these honest predictions; for a new case the four fold models' predictions are averaged.

Fold bagging creates OOF predictions and, for new cases, averages the predictions of the K fold models. The OOF vector gives (i) an honest internal validation score of the bagged model and (ii) leakage-free inputs for stacking. Averaging K models trained on overlapping $(K-1)/K$ subsets also lowers variance (Ch. 4). AutoGluon can repeat the fold split with different seeds (`num_bag_sets`) and average again.

Remark: a subtle mismatch

OOF predictions come from single fold models, while test predictions are averages of K fold models, which are slightly smoother. With K moderately large (5–10) the difference is small in practice.

4 Weighted Ensembles

Motivation. Models with similar accuracy often make different errors. Averaging them with weights chosen on held-out predictions reduces variance, and stacking learns the weights.

Example: three candidates

	linear	boosting	MLP
prediction	0.30	0.75	0.60
weight	0.10	0.60	0.30

$$\hat{y} = 0.10(0.30) + 0.60(0.75) + 0.30(0.60) = 0.66.$$

A weighted ensemble $\hat{y} = \sum_{m=1}^{M} w_m h_m(x)$ has one fixed weight per candidate, applied to every case. The weights are chosen to improve the internal validation (OOF) performance.

Algorithm: greedy ensemble selection (Caruana et al.)

Start from the empty ensemble. Repeat R times: add (with replacement) the candidate whose inclusion gives the best OOF score of the *averaged* ensemble. The weight of model m is (number of times selected)$/R$.

The weights are nonnegative and sum to 1; a model can be selected repeatedly, and useless models get weight 0. Selecting with replacement keeps the procedure from overfitting the validation set as easily as an unconstrained regression of y on z.

Theorem: ambiguity decomposition

For weights $w_m \geq 0$, $\sum_m w_m = 1$, and $\hat{y} = \sum_m w_m h_m$, under squared loss

$$(y - \hat{y})^2 = \sum_m w_m (y - h_m)^2 - \sum_m w_m (h_m - \hat{y})^2.$$

Hence the ensemble's error is never larger than the weighted average of the members' errors, and the improvement equals the weighted spread ("ambiguity") of the members' predictions.

Proof [Supplement]. $\sum_m w_m (y - h_m)^2 = \sum_m w_m \{(y - \hat{y}) + (\hat{y} - h_m)\}^2 = (y - \hat{y})^2 + 2(y - \hat{y}) \sum_m w_m (\hat{y} - h_m) + \sum_m w_m (\hat{y} - h_m)^2$, and the middle sum is $\hat{y} - \hat{y} = 0$. ■

Combining pays off when candidates are individually good *and* disagree; combining near-identical models gains nothing. The same convexity argument holds for any convex loss (e.g. log loss) via Jensen's inequality, without the exact decomposition.

5 How the Three Mechanisms Fit Together in AutoGluon

	Fold bagging	Stacking	Weighted ensemble
learning	same model on K folds	higher model g on candidates' OOF predictions z and original features x	repeatedly selects candidates that improve internal validation
new case	average of fold predictions	feeds candidates' predictions and original features to $g(x, z)$	sums candidates' predictions with fixed weights
prediction	$\frac{1}{K} \sum_{k=1}^{K} h^{(-k)}(x)$	$g(x, z)$	$\sum_{m=1}^{M} w_m h_m(x)$
core role	less prediction variance, OOF generation	learns new prediction rules from candidates' error patterns	combines the selected candidates into one final prediction

Fold bagging creates the OOF predictions needed by stacking; stacking and the weighted ensemble use them to combine the candidates' predictions. Stacking can be repeated in layers (`num_stack_levels`): the level-2 models are themselves fold-bagged, produce their own OOF predictions, and a final weighted ensemble combines the top layer. Because the original features are passed to each level ($g(x, z)$), a higher layer can still use them directly.

6 Setting the Search Space

```
predictor = TabularPredictor(label=label, eval_metric=metric)
predictor.fit(train_data, presets=..., time_limit=..., hyperparameters=...)
```

Setting	Role
`eval_metric`	metric used to rank candidates and to select the ensemble
`presets`	bundles the candidate portfolio, ensembling, refitting and cost settings (e.g. quality versus speed)
`time_limit`	approximate time budget for `fit()`
`hyperparameters`	the candidate portfolio: which model families and which settings of each
`num_bag_folds`	number of folds used to create OOF predictions
`num_stack_levels`	number of extra stacking layers that use lower models' predictions as inputs

The metric must match the decision of Ch. 1 (e.g. log loss for probability quality, PR-AUC for rare positives); otherwise the ensemble optimizes the wrong target. The time limit is part of the experiment: report it together with the leaderboard.

7 Responsible Use of AutoML

- AutoML automates model search and combination, not problem definition: the target, prediction time, leakage checks and split structure remain human decisions.
- Internal validation scores are already "used" for selection, so they are optimistic; the final number comes from a test set that AutoGluon never saw.
- Compare the ensemble with simple baselines (regularized logistic regression, one boosting model); the gain must justify the extra prediction time, memory and operational complexity.
- Inspect the leaderboard: fit time, prediction time, stack level, and the score variability of the top candidates.

Summary

- No single tabular model is always best; combining candidates is a decision in its own right.
- Stacking needs out-of-fold predictions; in-sample predictions reward memorization.
- Fold bagging creates OOF predictions and averages fold models for new cases; stacking learns $g(x, z)$; the weighted ensemble combines selected candidates with fixed convex weights.
- A convex combination's loss is at most the average member loss; gains come from diverse, individually strong candidates.
- AutoGluon manages fitting, validation and combination; the validation design and the final test remain the analyst's responsibility.